TP1

Conception et analyse des algorithmes

Nom et prénom : KAZALO Wilfride
Licence GL - Semestre 6 - Année académique 2025 - 2026
Cours : Conception et analyse des algorithmes - M. KARABOU

Partie 1 Définitions et concepts fondamentaux

1 Définition et exemples

Un algorithme est une suite finie d'instructions précises et non ambiguës permettant de résoudre un problème. Il transforme des données d'entrée en un résultat et doit se terminer pour toute entrée valide.

Exemple simple : trouver le plus grand de deux nombres a et b. On les compare ; si $a \geq b$, on retourne a, sinon on retourne b. Exemple complexe : calculer un itinéraire GPS entre deux lieux en tenant compte des routes et de leur durée de parcours, à l'aide d'un algorithme de plus court chemin.

2 Importance et critères de qualité

Un bon algorithme permet d'obtenir un résultat fiable avec un coût raisonnable. On peut retenir trois critères de qualité :

La correction : il doit fournir le résultat attendu pour toute entrée valide et terminer son exécution. Par exemple, un tri doit produire une suite croissante contenant exactement les éléments de départ, même en présence de doublons.

L'efficacité : il doit utiliser raisonnablement le temps de calcul et la mémoire. Par exemple, rechercher dans un grand tableau déjà trié par dichotomie demande beaucoup moins de comparaisons qu'un parcours complet.

La clarté : ses étapes doivent être compréhensibles et sans ambiguïté pour faciliter la programmation et la vérification. Par exemple, préciser les bornes d'une boucle et nommer une variable somme rend un calcul de moyenne plus facile à contrôler.

Partie 2 Importance des algorithmes

3 Cinq domaines essentiels

1) Optimisation des performances : les algorithmes réduisent le travail nécessaire. Par exemple, le tri fusion traite efficacement un grand ensemble de données grâce à son coût en $O(n \log n)$.

2) Sécurité informatique : les algorithmes cryptographiques protègent les informations et permettent de contrôler l'authenticité des échanges. Par exemple, AES sert à chiffrer des données confidentielles.

3) Gestion des données : les algorithmes de tri, de recherche et d'indexation facilitent l'organisation et l'accès aux informations. Par exemple, une recherche dichotomique retrouve un identifiant dans un tableau trié.

4) Réseaux et navigation : les algorithmes de graphes déterminent des chemins dans un réseau. Par exemple, Dijkstra permet de chercher le trajet de durée minimale entre deux lieux.

5) Intelligence artificielle : les algorithmes apprennent des régularités à partir d'exemples pour classer, reconnaître ou prédire. Par exemple, un réseau de neurones peut apprendre à distinguer des visages.

4 Sécurité et cryptographie

La cryptographie fournit des algorithmes pour préserver la confidentialité des données, détecter certaines modifications et vérifier une identité ou une signature. Ces fonctions reposent sur des mécanismes différents ; le chiffrement seul ne les assure pas toutes.

AES est un algorithme de chiffrement symétrique : une même clé secrète sert à chiffrer et à déchiffrer. Il traite des blocs de 128 bits avec une clé de 128, 192 ou 256 bits. Il transforme les données au moyen de plusieurs tours combinant substitutions, déplacements, mélanges et ajout de clés de tour dérivées de la clé secrète. Les transformations inverses permettent de retrouver le texte initial avec la bonne clé. Il peut ainsi protéger le contenu d'un fichier confidentiel dans un système de chiffrement adapté.

Référence pour AES : NIST, FIPS 197, Advanced Encryption Standard, 2023.

Partie 3 Complexité algorithmique

5 Définition et utilité

La complexité algorithmique décrit l'évolution des ressources nécessaires quand la taille n de l'entrée augmente. Elle permet de comparer les algorithmes et d'anticiper leur comportement sur de grandes données, indépendamment d'un temps mesuré sur une machine particulière. On distingue notamment le meilleur cas, le cas moyen et le pire cas.

6 Complexité en temps et en espace

a) Complexité en temps : nombre d'opérations élémentaires exécutées en fonction de la taille de l'entrée. b) Complexité en espace : quantité de mémoire nécessaire en fonction de cette taille. L'espace auxiliaire désigne la mémoire supplémentaire, en excluant les données d'entrée.

7 Calcul de la complexité en temps

Taille n	Opérations T(n)	T(n) / n²
2	10	2,5
5	50	2
10	200	2

Quand n passe de 5 à 10, il double et le nombre d'opérations passe de 50 à 200, donc il quadruple. Cela suggère une croissance quadratique : T(n) est approximativement proportionnel à n^2. La réponse attendue est $O(n^2)$, et même $\Theta(n^2)$ si cette croissance se confirme pour les grandes tailles.

La valeur pour n = 2 ne suit pas exactement la formule $2n^2$. Trois mesures seules ne prouvent pas une complexité asymptotique ; une preuve demanderait d'analyser les opérations de l'algorithme.

8 Calcul de la complexité en espace

Un tableau de n cases, contenant chacune une valeur de taille constante, occupe $\Theta(n)$ d'espace, donc $O(n)$. La mémoire augmente proportionnellement au nombre de résultats stockés.

Partie 4 Applications pratiques

9 Exemple du tri fusion

Le tri fusion, ou MergeSort, divise le tableau en deux parties, trie récursivement chaque partie, puis fusionne les deux parties triées en prenant à chaque étape le plus petit élément disponible. La division s'arrête lorsqu'une partie contient au plus un élément.

Exemple : [4, 1, 3, 2] est divisé en [4, 1] et [3, 2], puis en éléments isolés. Les premières fusions donnent [1, 4] et [2, 3] ; la dernière donne [1, 2, 3, 4]. La version classique prend $\Theta(n \log n)$ en temps et $\Theta(n)$ en espace auxiliaire. Elle convient au tri de grands ensembles d'enregistrements et peut être stable : les éléments de même clé conservent leur ordre initial.

10 Mémoire dans les systèmes embarqués

Les systèmes embarqués, comme les capteurs et les microcontrôleurs, disposent souvent de peu de mémoire vive. Un algorithme qui crée de grands tableaux temporaires peut épuiser cette mémoire et empêcher le programme de fonctionner. Il faut donc limiter les copies, réutiliser les espaces disponibles et choisir un algorithme adapté à la capacité du matériel, parfois au prix d'un temps de calcul plus long.

Partie 5 Résolution de problèmes

11 Navigation GPS avec Dijkstra

Pour aller d'un lieu A à un lieu D en ville, on représente les intersections par des sommets et les routes par des arêtes. Leur poids indique une distance ou une durée de trajet non négative. Une route à sens unique est représentée par une arête orientée.

Dijkstra initialise la distance de A à 0 et celles des autres sommets à l'infini. Il choisit ensuite le sommet non traité de distance provisoire minimale, puis améliore les distances de ses voisins si un trajet passant par lui est plus court. Il mémorise les prédécesseurs et recommence jusqu'à fixer la distance de D ou constater que D est inaccessible. Les prédécesseurs permettent de reconstruire le chemin.

Exemple : A-B = 2 min, A-C = 5 min, B-C = 1 min, B-D = 6 min et C-D = 2 min. Depuis A, on obtient d(B) = 2 et d(C) = 5. En traitant B, d(C) devient 3 et d(D) devient 8. En traitant C, d(D) devient 5. Le meilleur trajet est donc A → B → C → D, de durée 2 + 1 + 2 = 5 minutes.

Dijkstra convient à ces poids non négatifs. Pour une grande carte, A* peut orienter la recherche vers la destination au moyen d'une estimation du coût restant ; une heuristique adaptée conserve la garantie d'un chemin optimal.

Bonus Application en intelligence artificielle

12 Reconnaissance faciale

En intelligence artificielle, les algorithmes permettent d'apprendre un modèle à partir de données, puis de l'utiliser sur de nouveaux exemples. En reconnaissance faciale, FaceNet utilise un réseau de neurones convolutif qui transforme une image de visage en un vecteur numérique décrivant ses caractéristiques.

Pendant l'apprentissage, le modèle rapproche les vecteurs de visages d'une même personne et éloigne ceux de personnes différentes. Pour vérifier deux visages, on calcule la distance entre leurs vecteurs et on la compare à un seuil. Pour reconnaître une personne, on compare le vecteur obtenu à ceux des identités enregistrées. Cette représentation apprise permet de mieux gérer les variations d'apparence qu'une simple comparaison des pixels, sans supprimer les erreurs possibles.

Référence : Schroff, Kalenichenko et Philbin, FaceNet A Unified Embedding for Face Recognition and Clustering, CVPR, 2015.